SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

SSAFY

Django

Model

Python

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Model

- Model class

Model Field

- Field types
- Field options

Migrations

- 추가 Migrations

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

목 차

Admin site

- 관리자 인터페이스

참고

- 데이터베이스 초기화
- Migrations 관련
- SQLite

학습 시작

Django는 파이썬 문법으로 테이블을 만들고 관리합니다.

1. 파이썬 객체지향 문법으로 간단한 자유 게시판 웹앱 DB 테이블을 만들고 수정합니다.
2. 관리자 화면에서 글을 직접 써 보고 지워 보면서 CRUD의 흐름을 몸으로 익힐 수 있습니다.

- 이번 시간이 끝나면, Python 클래스에서 DB 스키마로 자동 변환하는 원리를 이해합니다.
- **모델 변경 이력**을 안전하게 관리하고 Admin Site 를 활용해 코드 한 줄 작성 없이 데이터 CRUD를 수행할 수 있습니다.

Objectives

학습 목표

- ✓ Django Model 클래스를 이용해 데이터베이스 테이블 구조를 설계하고 “설계도(blueprint)” 역할을 설명할 수 있다.
- ✓ 다양한 Field type과 Field option을 지정해 컬럼의 데이터 유형과 제약 조건을 정의할 수 있다.
- ✓ python manage.py makemigrations / migrate 두 단계로 모델 변경 사항을 안전하게 데이터베이스에 반영할 수 있다.
- ✓ admin.site.register()로 모델을 Django Admin에 등록하여 웹 UI에서 게시글을 생성, 조회, 수정, 삭제(CRUD)할 수 있다.
- ✓ Django 기본 DB 엔진인 SQLite의 특성과 db.sqlite3를 Git에서 제외해야 하는 이유를 설명할 수 있다.

Model

Model이란?

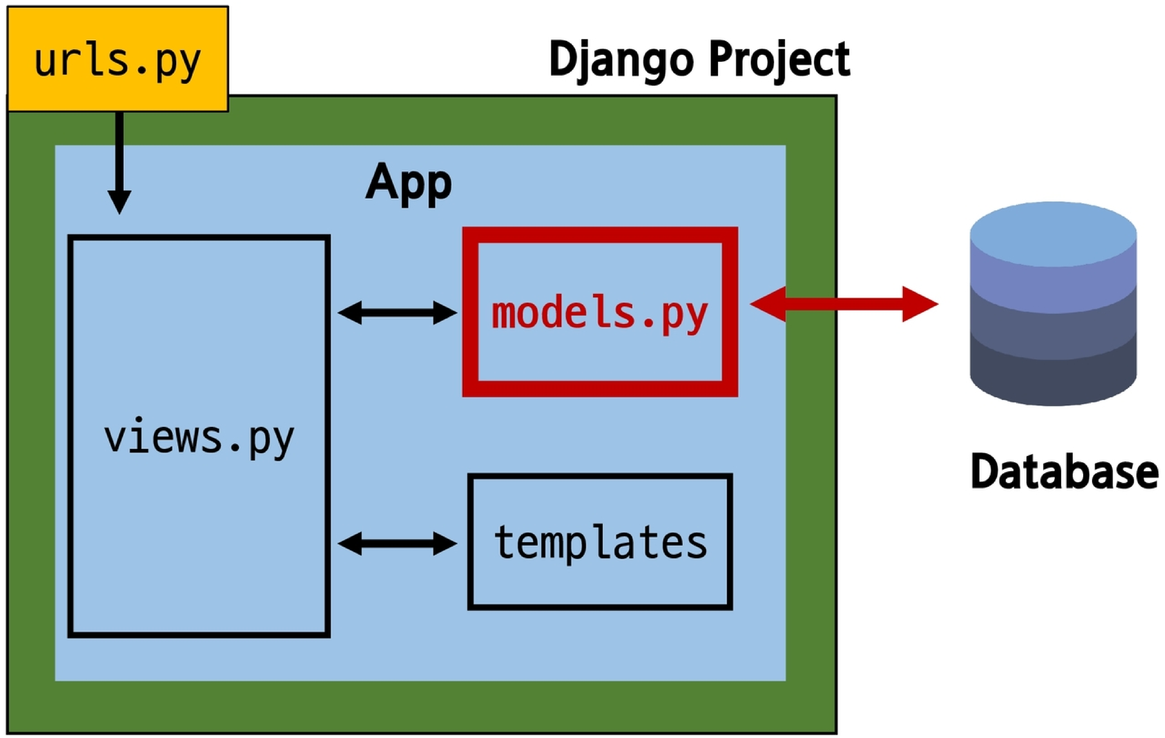
Model

데이터베이스와 Python 클래스(**객체**)로 추상화된 형태로 상호작용

- Django의 강력한 기능: 개발자가 데이터베이스에 대한 깊은 지식 없이도 쉽게 데이터 관리 가능
- 유지보수 및 확장성 증대: 데이터베이스 변경 시에도 코드 수정 최소화, 재사용 가능한 데이터 모델을 통해 개발 효율성 향상

Model을 통한 DB(데이터베이스) 관리

- **urls.py**: 사용자 요청의 시작점
- **views.py**: 요청을 처리하고 models.py를 통해 데이터를 다룸
- **models.py**: **데이터베이스**를 정의하고, **데이터베이스**와 상호작용
- **templates**: views.py로부터 받은 데이터를 사용자에게 보여줄 화면을 구성



<그림1_Django_Model_프로젝트 구조>

Model class

Model class란?

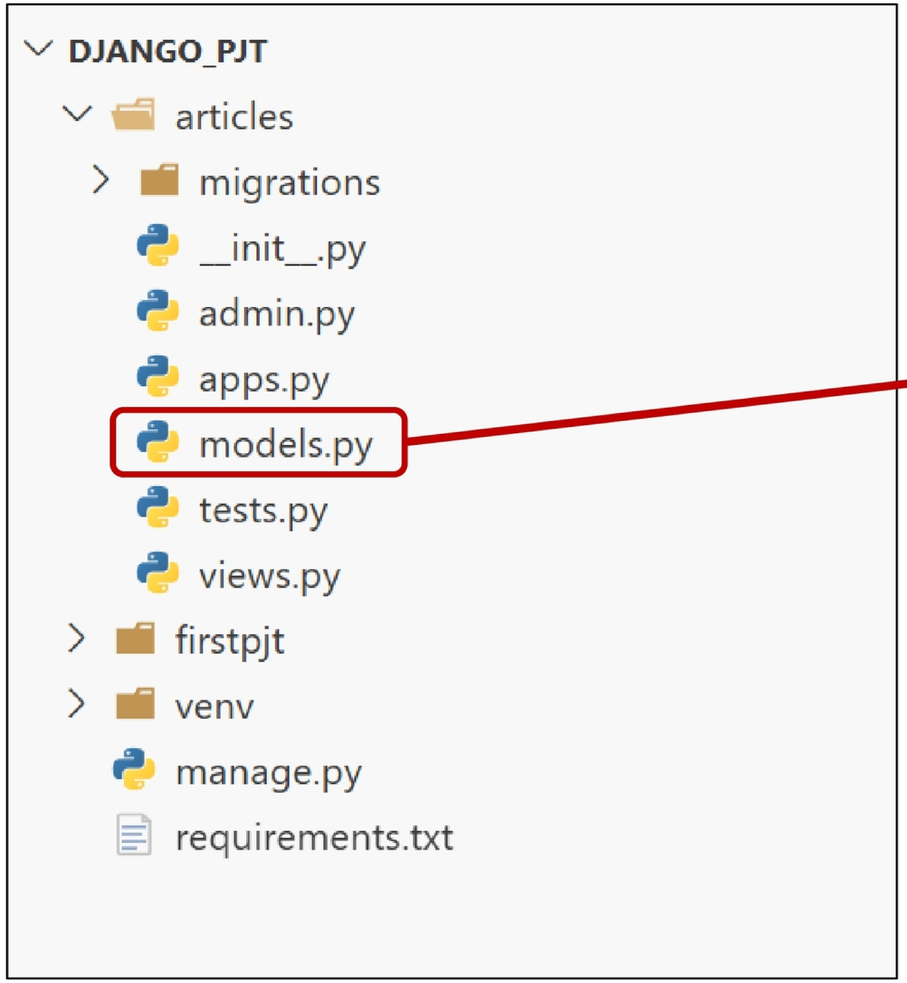
Model class

DB의 테이블을 **정의**하고
데이터를 조작할 수 있는 기능들을 제공

Model class는 데이터베이스 테이블의 구조를 설계하는 '청사진(blueprint)' 역할을 합니다.

어떤 데이터(컬럼)를 저장할지, 그 데이터의 형태는 어떠할지(타입, 길이 등)를 Python 코드로 명확히 정의합니다.

Model 클래스 작성



<그림2_Django_Model_프로젝트 구조2>

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

- articles 앱의 models.py에 위 코드를 작성합니다.

Model 클래스 살펴보기 (1/4)

- 작성한 모델 클래스는 최종적으로 DB에 다음과 같은 테이블 구조를 만듦

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

‘모델 클래스 == 테이블 설계도’

id	title	content
..	..	..
..	..	..

<표1_Django_Model_예시 테이블>

※ id 필드는 Django가 자동 생성

Model 클래스 살펴보기 (2/4)

- `django.db.models` 모듈의 Model이라는 부모 클래스를 상속받음
  - Model은 `model`에 관련된 모든 코드가 이미 작성 되어있는 Class
    - 개발자는 가장 중요한 테이블 구조를 어떻게 설계할지에 대한 코드만 작성하도록 하기 위한 것 ("상속"을 활용한 프레임워크의 기능 제공)
  - "약 2,000줄짜리 클래스" 한 줄 추가 ➔ "우리가 추가로 쓴 건 단 몇 줄, Django가 써둔 건 2,000줄"

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

`models.Model` (Django가 만들어둔 부모 클래스)
- 테이블 자동 생성 (Migrations)
- SQL 자동 변환
- 데이터 저장 / 조회 / 수정 / 삭제
- 데이터 유효성 검사
- id 필드 자동 생성

https://github.com/django/django/blob/main/django/db/models/base.py

프레임워크 없이 직접 만든다면?

- "DB 연결, SQL 작성, 커밋은 `models.Model`이 대신 ➔ 개발자는 테이블 구조 설계에만 집중"

```python
import sqlite3

conn = sqlite3.connect('db.sqlite3')
cursor = conn.cursor()
cursor.execute('''
    CREATE TABLE IF NOT EXISTS article (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        title VARCHAR(10) NOT NULL,
        content TEXT NOT NULL
    )
''')
conn.commit()
conn.close()
```

순수 Python으로 작성하려면

```python
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

프레임워크의 도움을 받으면

이것이 프레임워크!

- 코드 한 줄에 담긴 두 가지 "가져다 쓰기"
  - `models.Model` ➔ 상속으로 테이블 관련 기능 전체를 물려받음
  - `models.CharField` ➔ 이것도 Django가 만들어둔 클래스. "문자열이고 최대 10자"라고 선언만 하면 DB 컬럼 타입 결정, 길이 검사는 CharField가 처리

```
class Article(models.Model):  # 1) 클래스를 "상속"받아 기능을 물려받음
    title = models.CharField(max_length=10)  # 2) 만들어진 클래스를 "가져다 씀"
```

- 사고의 전환
  - 알고리즘 사고: "이 기능을 어떻게 직접 만들지?"
  - 프레임워크 사고: "이 기능은 이미 있다. 내 상황에 맞게 어떻게 조합하지?"

➢"앞으로 배울 Form 등 여러 기능들도 모두 Django가 제공하는 클래스를 상속받는 같은 패턴입니다."

Model 클래스 살펴보기 (3/4)

- 클래스 변수명
  - 테이블의 각 “필드(열) 이름”

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

id	title	content
..	..	..
..	..	..

<표1_Django_Model_예시 테이블 재활용>

Model 클래스 살펴보기 (4/4)

- Model Field
  - 데이터베이스 테이블의 열(column)을 나타내는 중요한 구성 요소
  - “데이터의 유형”과 “제약 조건”을 정의

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

유효성 검사
정해진 규칙이나 조건에 따라 데이터의 정확성을 확인하는 과정

<개념1_Django_Model_유효성 검사>

TIP

- `CharField`의 `max_length` 지정은 SQLite에서는 선택 사항입니다.
- SQLite는 문자열 길이를 DB 차원에서 강제하지 않기 때문에, 길이 검사는 Django의 유효성 검사가 담당합니다.
- 다른 DB(MySQL 등)에서는 필수이므로, 항상 명시하는 습관을 들이세요.

Model Field

Model Field이란?

Model Field

DB 테이블의 **필드(열)** 정의

데이터 타입 및 **제약 조건** 명시

Field types Field options

Django는 이러한 필드 정의를 바탕으로 데이터베이스 컬럼을 자동으로 생성하고, 데이터 입력 시 유효성 검사 등 필요한 기능을 제공합니다. 정확한 필드 정의는 애플리케이션의 안정성을 높이는 데 필수적입니다.

Model Field

1. Field types (필드 유형)
   - 데이터베이스에 저장될 “데이터의 종류”를 정의
2. Field options (필드 옵션)
   - 필드의 “동작”과 “제약 조건”을 정의

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

Field types

Field types

Field Types

데이터베이스에 저장될 “데이터의 종류”를 정의

➢ CharField, TextField도 Django가 만들어둔 클래스
우리는 클래스를 가져다 쓰면서 옵션(max_length 등)만 지정

```
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

Field types

CharField()

제한된 길이의 문자열을 저장

- `max_length`는 SQLite에서는 생략 가능하지만, 명시하는 것을 권장
  - 예: 이름, 제목, 이메일처럼 짧고 길이를 예상할 수 있는 문자열

Field types

TextField()

길이 제한이 없는 대용량 텍스트를 저장
(무한대는 아니며 사용하는 시스템에 따라 달라짐)

CharField vs TextField, 언제 무엇을 쓸까?

구분	CharField	TextField
길이	짧고 예상 가능	길거나 예상하기 어려움
max_length	지정 (권장)	지정하지 않음
예시	이름, 제목, 이메일	게시글 본문, 상품 설명

TIP

- 길이가 255자를 넘을 것 같다면 TextField를 먼저 고려하세요.

주요 필드 유형

- 문자열 필드
  - `CharField`, `TextField`
- 숫자 필드
  - `IntegerField`, `FloatField`
- 날짜/시간 필드
  - `DateField`, `TimeField`, `DateTimeField`
- 파일 관련 필드
  - `FileField`, `ImageField`

TIP

다양한 Django 모델 필드 유형을 모두 다루기는 어렵습니다.

필요한 필드 유형은 **Django 공식 문서**를 참고하여 찾아 사용해야 합니다.

https://docs.djangoproject.com/en/5.2/ref/models/fields/#field-types

Field options

Field options이란?

Field Options

필드의 "동작"과 "제약 조건"을 정의

```
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

제약 조건이란?

제약 조건

Constraint

특정 규칙을 강제하기 위해 테이블의 열이나 행에
적용되는 규칙이나 제한사항

숫자만 저장되도록 제한을 두거나,
문자가 100자 까지만 저장되도록 하는 등의 제한 조건을 의미합니다.

주요 필드 옵션

- null
  - 데이터베이스에서 `NULL` 값을 허용할지 여부를 결정 (**기본값**: `False`)
- blank
  - `form`에서 빈 값을 허용할지 여부를 결정 (**기본값**: `False`)
- default
  - 필드의 기본값을 설정

※ **DB에서 제약조건을 설정하는 방법은 DB 챕터에서 학습할 예정**

TIP

다양한 Django 모델 필드 유형만큼이나 필드 옵션 또한 매우 다양합니다. **각 모델 필드가 어떤 옵션**을 가질 수 있는지는 Django 공식 문서를 참고하여 필요에 맞게 찾아 사용하시길 권장합니다.

https://docs.djangoproject.com/en/5.2/ref/models/fields/#field-options

Migrations

Migrations이란?

Migrations

model 클래스의 변경사항(**필드 생성, 수정 삭제** 등)을
DB에 최종 반영하는 방법

모든 변경 사항이 코드로 관리되어 협업 시 모델 변경 내역에 대한 추적과 공유가 수월합니다.

Django 모델 클래스와 Migration 과정 (1/2)

- 모델 정의부터 마이그레이션 파일 생성까지

```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

```
$ python manage.py makemigrations
```

```
$ python manage.py makemigrations

Migrations for 'articles':
  articles/migrations/0001_initial.py
    + Create model Article
```

모델 클래스 작성/수정

Django에서는 Python 클래스로 정의합니다.

- `CharField`: 제한된 길이의 문자열 저장
- `TextField`: 길이 제한 없는 대용량 텍스트

마이그레이션 파일 생성 명령어

Python 코드로 된 migration 파일(설계도)를 만드는 방법

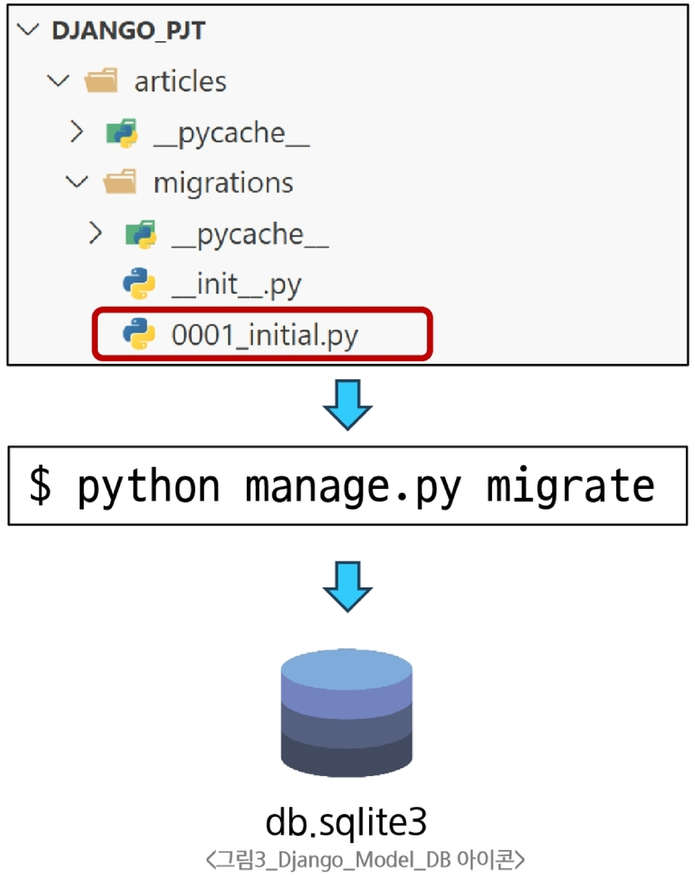
- 모델 변경을 감지하고 migration 파일을 생성하는 명령어
- migration 파일 == 최종 설계도

Django 모델 클래스와 Migration 과정 (2/2)

- 생성된 최종 설계도(`0001_initial.py`) 마이그레이션 파일을 DB에 반영하기

SQL
DB를 조작하는데 사용되는 표준 질의 언어

<개념2_Django_Model_SQL>



<그림3_Django_Model_DB 아이콘>

마이그레이션 파일(`0001_initial.py`)

- 마이그레이션 파일은 모델 변경사항을 기록한 Python 코드

※ 마이그레이션 파일은 DB 테이블 변경 내역을 순차적으로 저장하여 추적 관리합니다.

SQL 실행 (자동 변환)

- `migrate` 명령어는 마이그레이션 파일의 Python 코드를 SQL 문으로 자동 변환
- 변환 과정
  - Python 코드 → 번역 → SQL 쿼리문 →데이터베이스 실행

Migrations 과정 핵심 정리

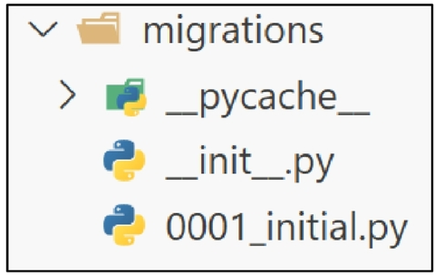
```python
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
```

model class

(설계도 초안)

makemigrations

<그림5_Django_Model_마이그레이션 0001-2>



migration 파일(0001_initial.py)

(최종 설계도)

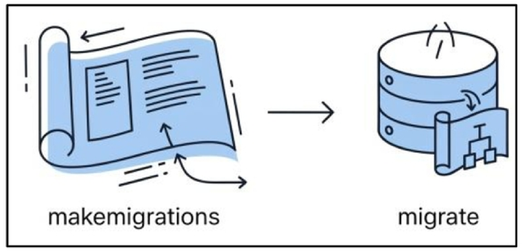
migrate

db.sqlite3

(DB)

<그림3_Django_Model_DB 아이콘 재사용>

Migrations 핵심 명령어 2가지



<그림6_Django_Model_마이그레이션 이미지>

```
$ python manage.py makemigrations
```

- 역할: model class를 기반으로 최종 설계도(migration)을 작성

```
$ python manage.py migrate
```

- 역할: 최종 설계도를 DB에 전달하여 반영

TIP

- 마이그레이션 파일은 되도록이면 직접 건드리지 마세요!
- migrations 폴더에 자동으로 생성된 파일들은 직접 수정하거나 삭제하지 않는 것이 원칙입니다.

Migration 경고 메시지

- Django 서버를 시작할 때 출력되는 migration 관련 경고 메시지
  - 이 메시지는 프로젝트 실행 시 **미적용 마이그레이션 파일**이 있을 때 나타납니다.

```
$ python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
```

- 해결방법
  - 경고 메시지에 명시된 대로 `python manage.py migrate` 명령어를 실행
- 중요한 이유
  - 최신 모델 변경 사항이 데이터베이스에 반영되지 않아 **프로젝트가 제대로 작동하지 않을 수 있음**

추가 Migrations

이미 생성된 테이블에 필드를 추가해야 한다면?

- 이미 기존 테이블이 존재하는 경우, 새 필드를 추가할 때 문제가 발생할 수 있음
  - 기존에 레코드가 있는 테이블에 새로운 필드를 추가하면 어떤 값으로 채울지 결정해야 함
  - Django의 `makemigrations` 실행 시 기본값 설정을 요구하는 프롬프트가 표시됨

id	title	content	created_at	updated_at
..	..	..	..	..
..	..	..	..	..
..	..	..	..	..

<표2_Django_Model_필드 추가 테이블>

Migration 과정 - 추가 모델 필드 작성 (1/5)

- 새로운 필드 작성

Model Field
models.필드 유형(필드 옵션)

<개념3_Django_Model_모델 필드 형태 리마인드>

모델 클래스 수정

Django 기존 모델을 수정합니다.

```
# articles/models.py
class Article(models.Model):
    title = models.CharField(max_length=10)
    content = models.TextField()
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
```

DateTimeField의 필드 옵션 (optional)

auto_now

데이터가 저장될 때마다

자동으로 현재 날짜시간을 저장

auto_now_add

데이터가 처음 생성될 때만

자동으로 현재 날짜시간을 저장

Migration 과정 - 추가 모델 필드 작성 (2/5)

- 새로운 필드 추가 후 **makemigrations** 명령어 입력 `$ python manage.py makemigrations`
  - 이미 기존 테이블이 존재하기 때문에 필드를 추가 할 때 필드의 기본 값 설정이 필요
  - **1번은 현재 대화를 유지하면서 직접 기본 값을 입력 하는 방법**
  - 2번은 현재 대화에서 나간 후 models.py에 기본 값 관련 설정을 하는 방법

```
It is impossible to add the field 'created_at' with 'auto_now_add=True' to article without providing a default.
This is because the database needs something to populate existing rows.
 1) Provide a one-off default now which will be set on all existing rows
 2) Quit and manually define a default value in models.py.
Select an option:
```

TIP

옵션 1: 일회성 기본값 제공

- 현재 대화형 프롬프트에서 기본값을 직접 입력합니다.
- 입력한 기본값은 기존의 모든 데이터 행에 적용됩니다.

Migration 과정 - 추가 모델 필드 작성 (3/5)

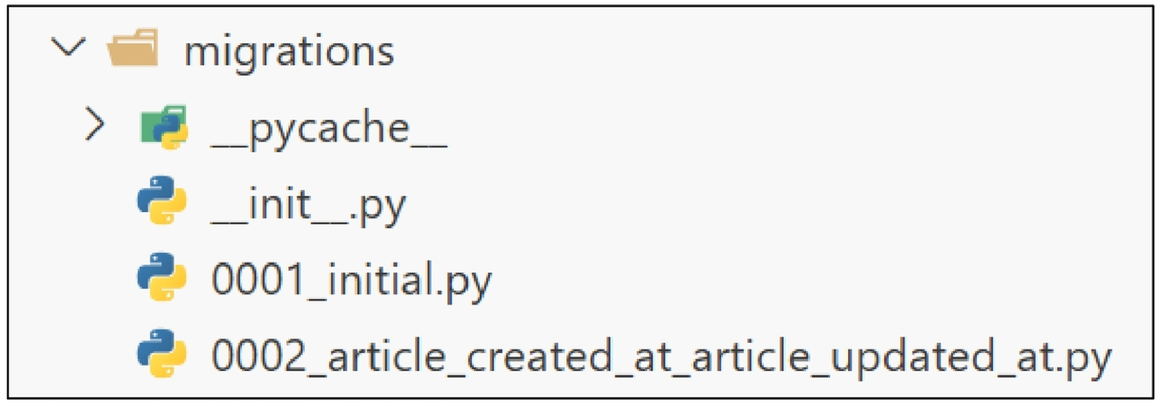
- 추가하는 필드의 기본 값을 입력해야 하는 상황
- 날짜 데이터이기 때문에 직접 입력하기 보다 Django가 제안하는 기본 값을 사용하는 것을 권장
- 아무것도 입력하지 않고 enter를 누르면 Django가 제안하는 기본 값으로 설정 됨

```
Please enter the default value as valid Python.
Accept the default 'timezone.now' by pressing 'Enter' or provide another value.
The datetime and django.utils.timezone modules are available, so it is possible to
provide e.g. timezone.now as a value.
Type 'exit' to exit this prompt
[default: timezone.now] >>>
```

Migration 과정 - 추가 모델 필드 작성 (4/5)

- migrations 과정 종료 후 2번째 migration 파일이 생성됨을 확인
- 이처럼 Django는 설계도를 쌓아가면서 추후 문제가 생겼을 시 복구하거나 되돌릴 수 있도록 함
  - (마치 ‘`git commit`’과 유사)

```
Migrations for 'articles':
  articles\migrations/0002_article_created_at_article_updated_at.py
    + Add field created_at to article
    + Add field updated_at to article
```

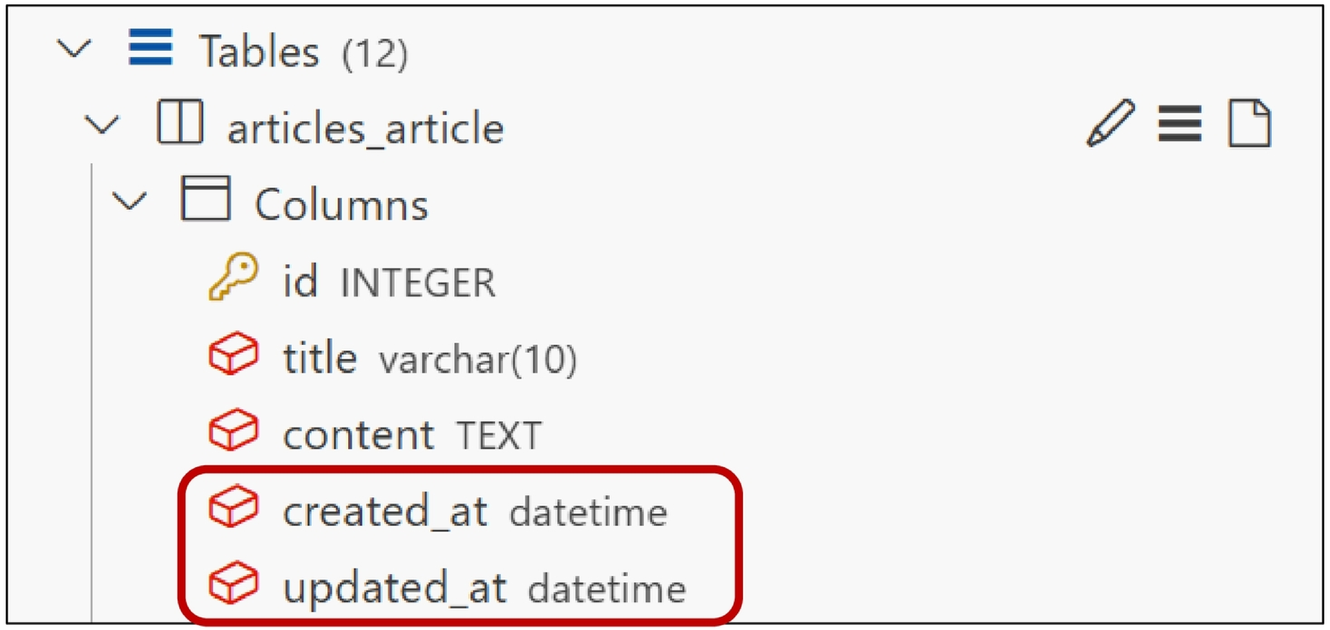


<그림7_Django_Model_Articles 마이그레이션 2>

Migration 과정 - 추가 모델 필드 작성 (5/5)

- migrate 후 테이블 필드 변화 확인

```
$ python manage.py migrate
```



<그림8_Django_Model_Articles 테이블 결과>

언제 Migration이 필요할까?

model class에 변경사항(1)이 생겼다면,

반드시 새로운 설계도를 생성(2)하고,

이를 DB에 반영(3)해야 한다.

(1) model class 생성/수정 → (2) makemigrations → (3) migrate

Admin site

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

관리자 인터페이스

Django가 제공하는 관리자 인터페이스

관리자 인터페이스

Automatic admin interface

Django가 추가 설치 및 설정 없이 자동으로 제공하는 관리자 인터페이스

- Django 관리자 인터페이스는 추가 설정 없이 자동 생성되는 웹 기반 관리 도구입니다.
- 주요 기능: 데이터베이스 모델의 **CRUD(생성, 읽기, 업데이트, 삭제)** 작업을 간편하게 수행할 수 있습니다.
- 활용: 빠른 **프로토타이핑**, **비개발자 데이터 관리, 내부 시스템 구축**에 이상적입니다.

1. Django admin 계정 생성

1. 터미널 열기

- Django 프로젝트 폴더로 이동해서 터미널을 엶 (**manage.py** 파일이 있는 위치에서 진행하면 됨)

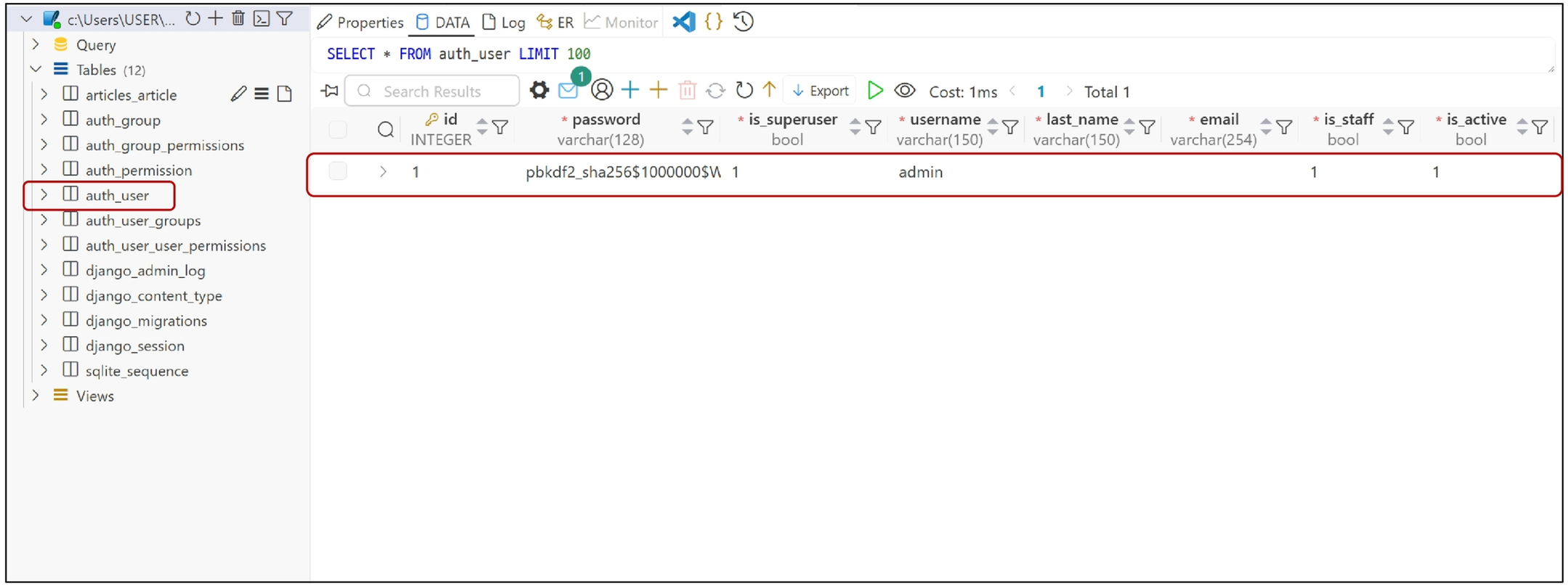
2. 관리자 계정 생성 명령어 입력

```
$ python manage.py createsuperuser
```

3. 정보 입력

- 사용자 이름(**username**): 관리자 페이지에 로그인할 때 사용할 아이디를 입력
- 이메일(**email address**): 선택 사항이라 입력하지 않고 엔터를 눌러 넘어가도 됨
- 비밀번호(**password**): 로그인할 때 사용할 비밀번호 입력**(비밀번호 입력 시 콘솔 창에 아무것도 나타나지 않음)**
- 비밀번호 확인(**password(again)**): 비밀번호를 한 번 더 입력해서 확인

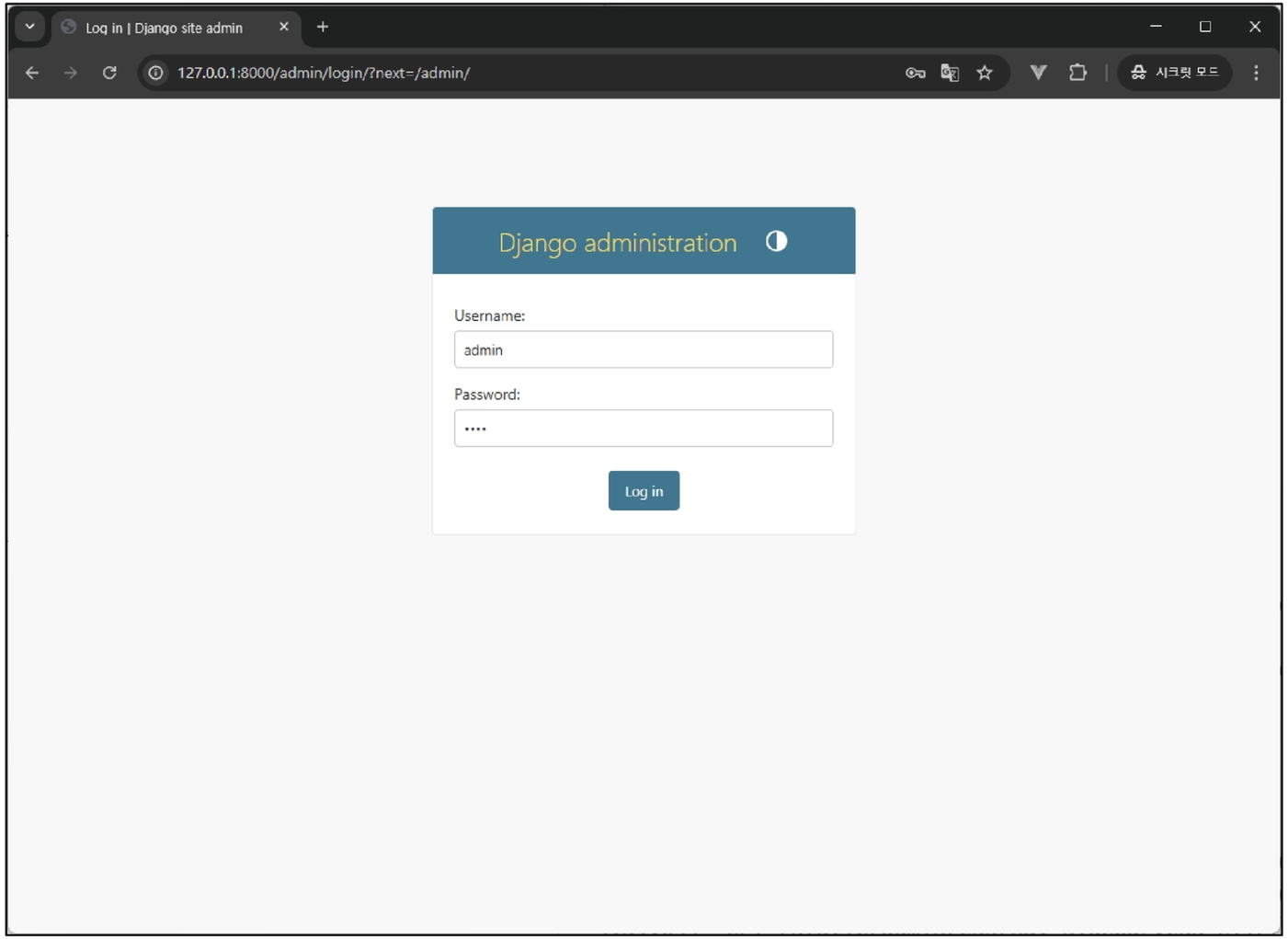
2. DB에 생성된 admin 계정 확인



<그림8_Django_Model_Articles 어드민 생성 결과>

3. 관리자 인터페이스 페이지

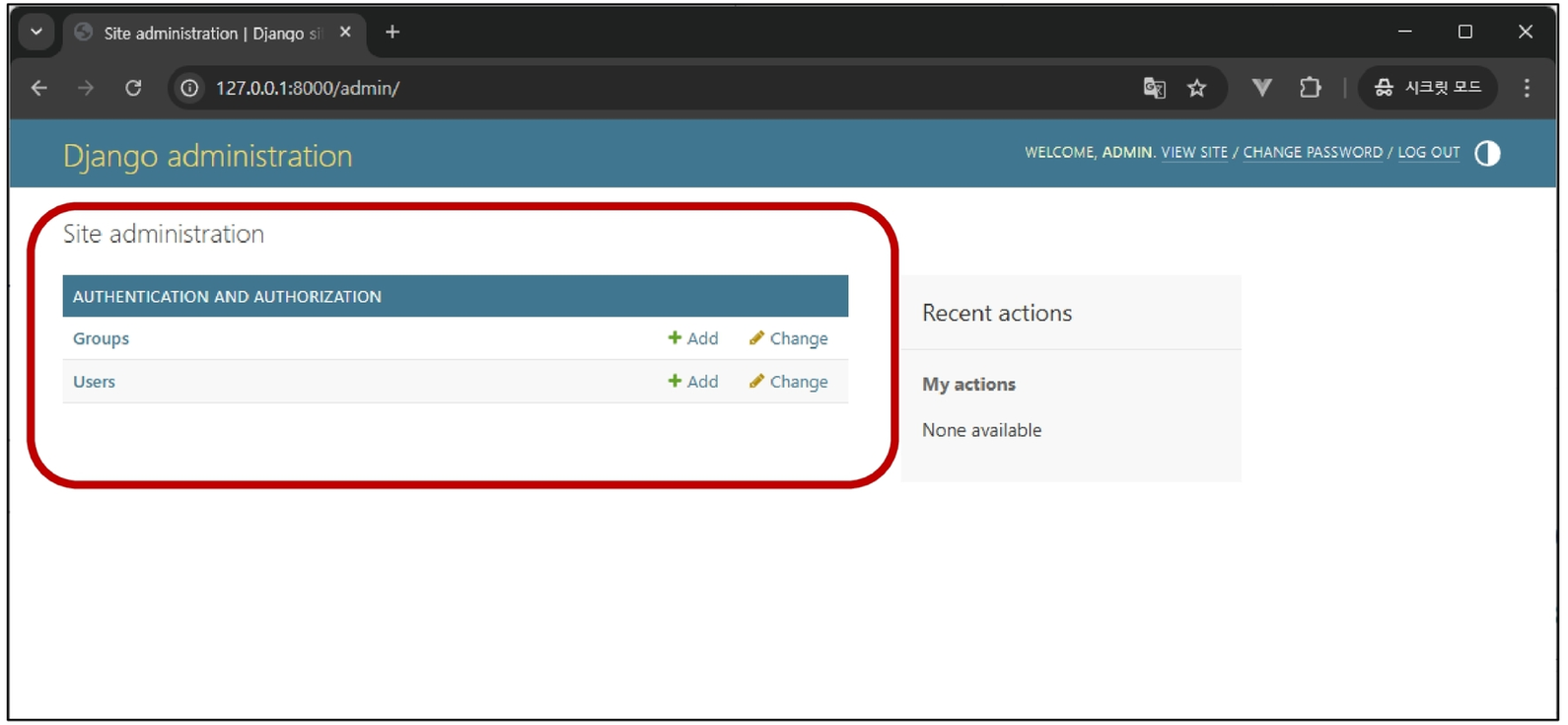
- 웹 브라우저를 열고 다음 주소로 이동
  - http://127.0.0.1:8000/admin
  - 생성한 관리자 계정으로 정상 로그인 확인



<그림9_Django_Model_Articles 어드민 로그인>

4. 관리자 인터페이스 접속 확인

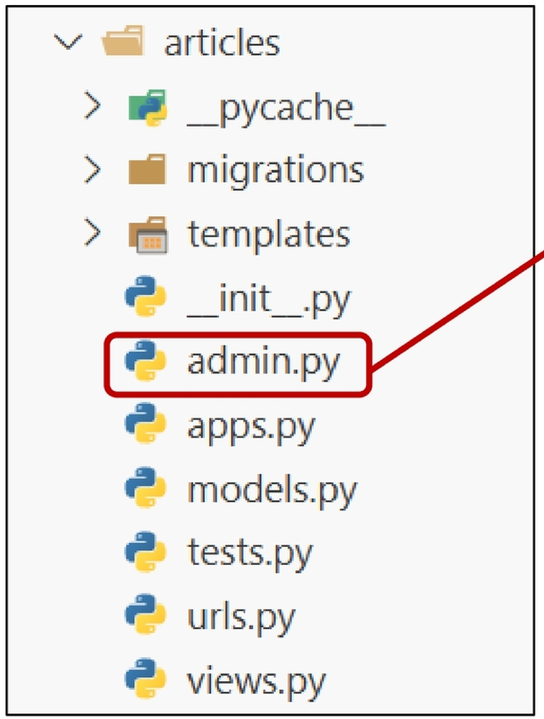
- 로그인 후 아무것도 보이지 않는 것이 정상!
  - 우리가 만든 모델을 관리하려면 **추가 설정이 필요**



<그림10_Django_Model_Articles 어드민 메인>

5. Admin site 모델 클래스 등록 및 확인(1/2)

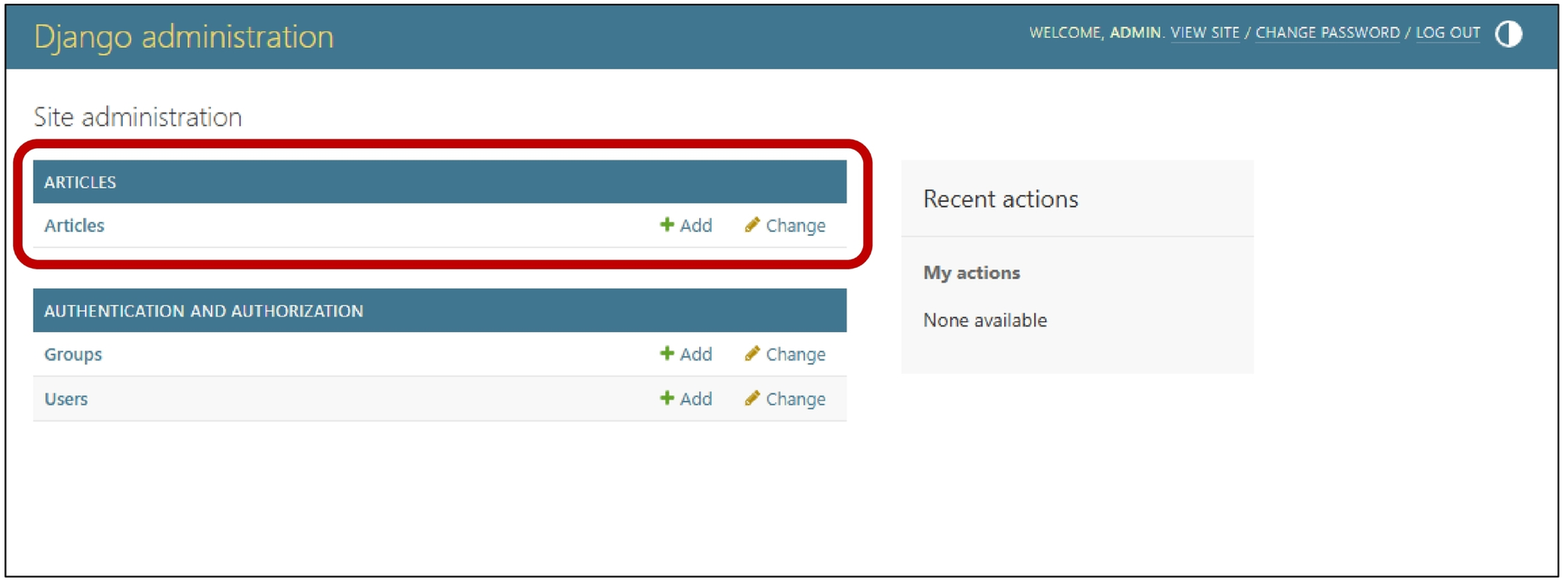
- admin.py에 작성한 모델 클래스를 등록해야만 admin site에서 확인 가능




```python
# articles/admin.py
from django.contrib import admin
from .models import Article

admin.site.register(Article)
```

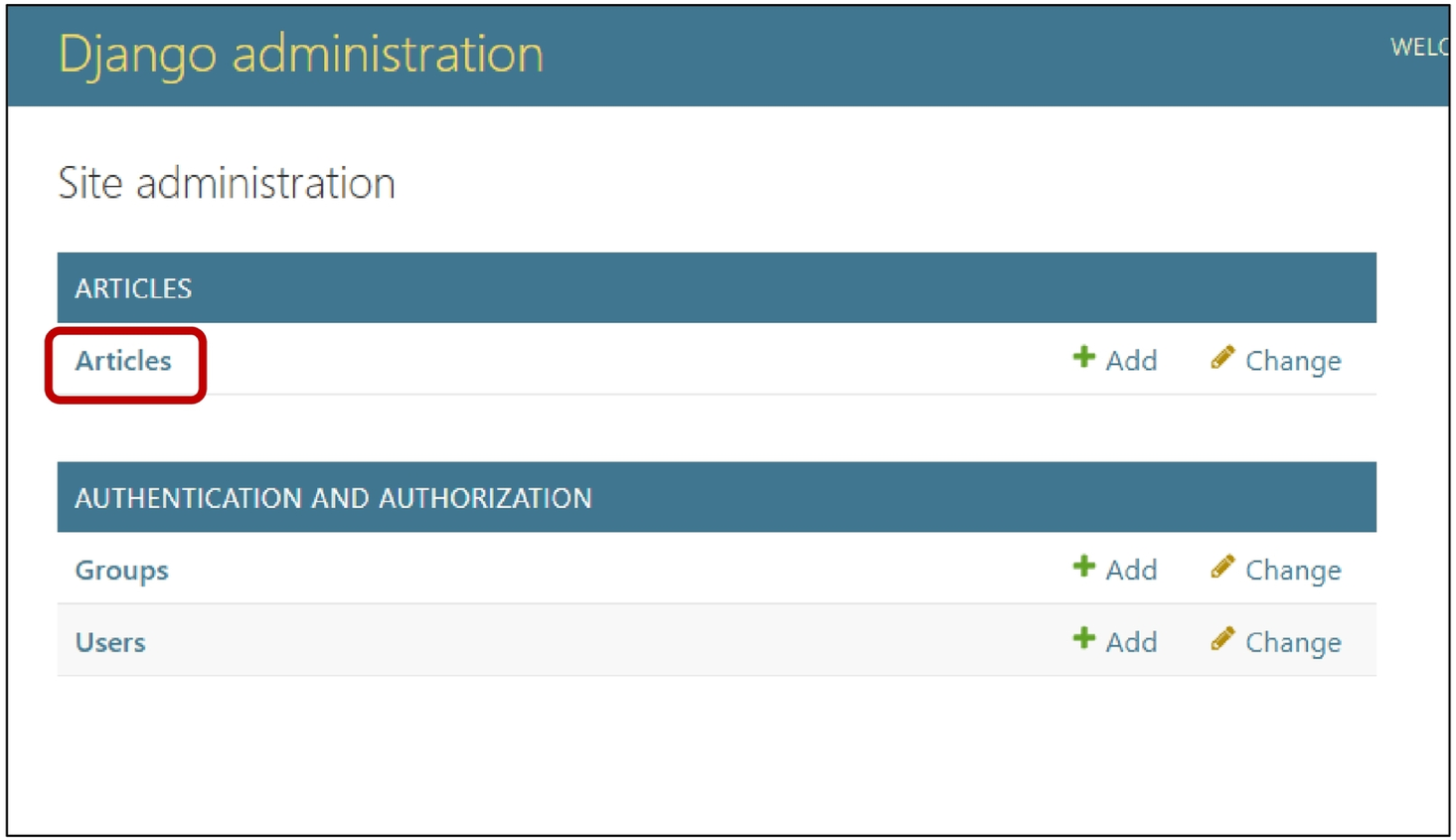
5. Admin site 모델 클래스 등록 및 확인(2/2)



<그림10_Django_Model_Articles 어드민 메인 아티클>

6. 데이터 생성, 수정, 삭제 테스트(1/4)

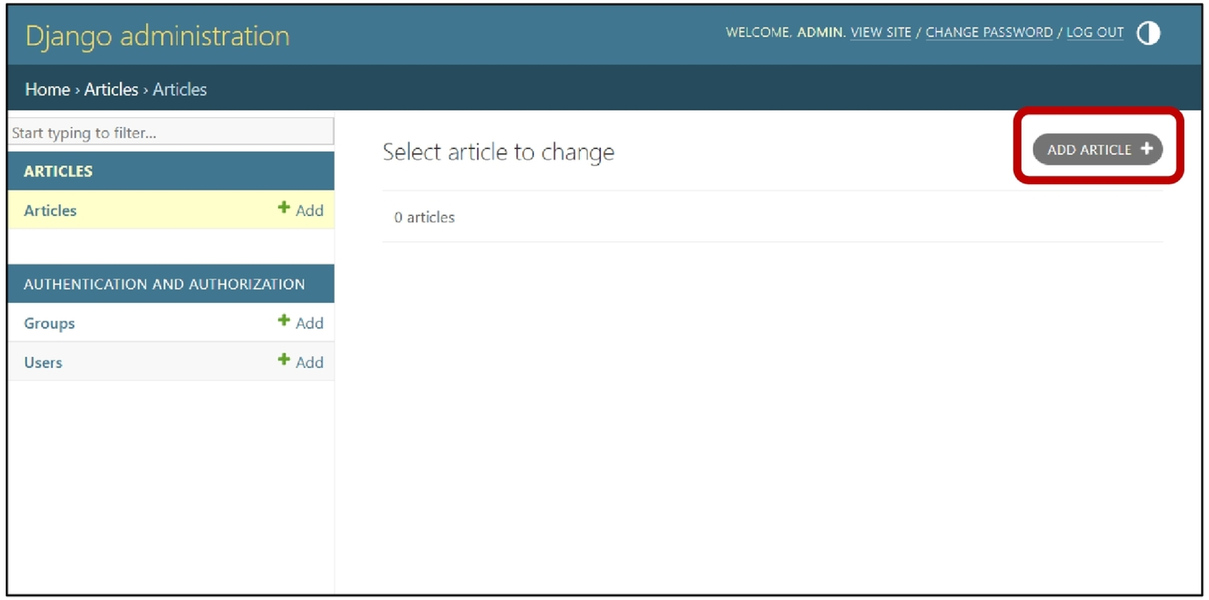
- Articles 앱에서 article에 대한 CRUD(생성, 읽기, 업데이트, 삭제) 를 위한 페이지로 이동



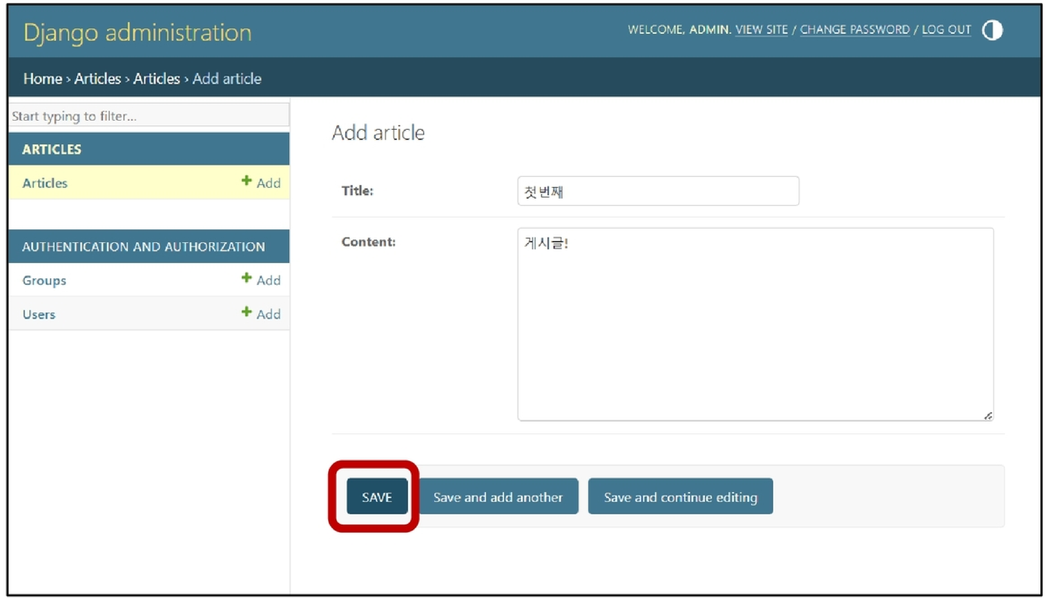
<그림11_Django_Model_Articles 어드민 패널 아티클 스몰>

6. 데이터 생성, 수정, 삭제 테스트(2/4)

- 새로운 Article **생성**



<그림12_Django_Model_Articles 어드민 패널 아티클 메인>

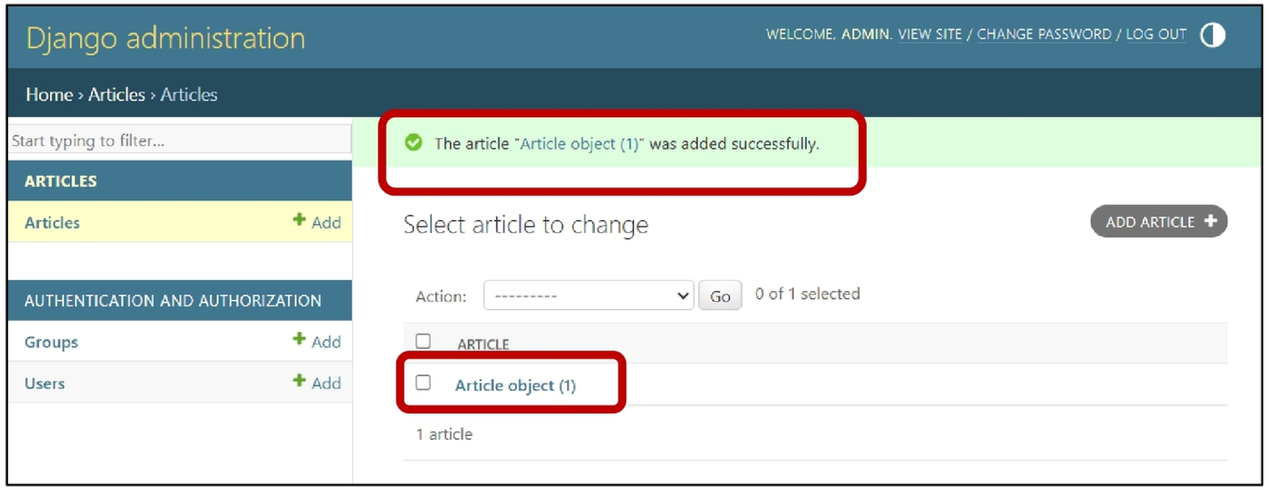


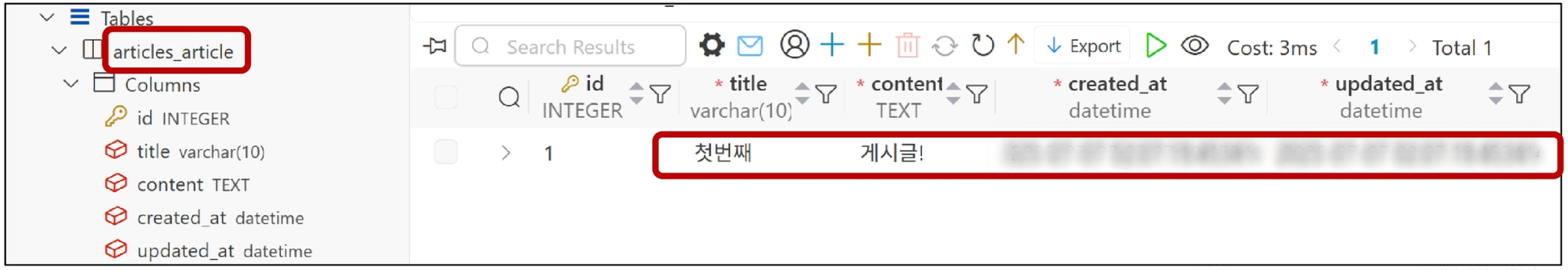
<그림13_Django_Model_Articles 어드민 아티클 crud>

6. 데이터 생성, 수정, 삭제 테스트(3/4)

- 성공 메시지와 함께 등록된 Article이 모두 조회됨
- 데이터베이스에서도 확인할 수 있음

<그림14_Django_Model_Articles 어드민 아티클 생성 성공>

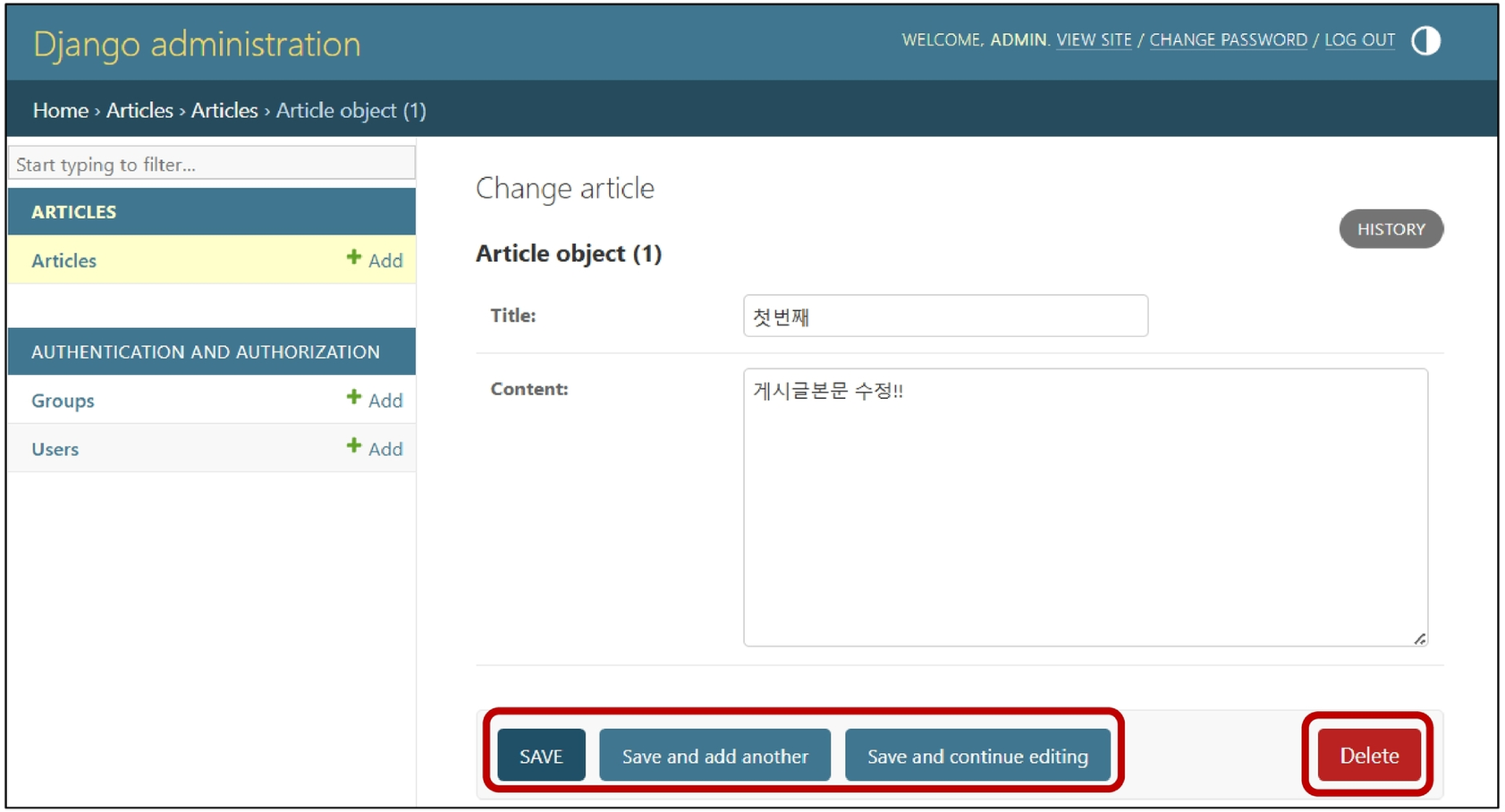




<그림15_Django_Model_Articles 데이터베이스 확인>

6. 데이터 생성, 수정, 삭제 테스트(4/4)

- 상세 페이지 내에서 원하는 데이터를 수정 또는 삭제할 수 있음



<그림16_Django_Model_Articles 어드민 수정 삭제>

참고

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

데이터베이스 초기화

데이터베이스 초기화 방법

1. Migration 파일 삭제
   - `makemigrations` 명령어로 생성되는 설계도(Migration 파일)

     ⊗ **아래 파일과 폴더는 삭제 금지**
     - **_ _ init _ _.py**
     - **migrations 폴더**

2. db.sqlite3 파일 삭제

<그림16_Django_Model_데이터베이스 초기화 파일 트리>

Migrations 관련

Migrations 기타 명령어

- showmigrations
  - migrations 파일들이 migrate 됐는지 안됐는지 여부를 확인하는 명령어
  - [X] 표시가 있으면 migrate가 완료되었음을 의미

```
$ python manage.py showmigrations
```

- sqlmigrate
  - 해당 migrations 파일이 SQL 언어(DB에서 사용하는 언어)로 어떻게 번역 되어 DB에 전달되는지 확인하는 명령어
  - python manage.try sqlmigrate <앱 이름> <마이그레이션 이름>

```
$ python manage.py sqlmigrate articles 0001
```

최초 migrate 시 출력 내용이 많은 이유는?

- Django 프로젝트가 동작하기 위해 미리 작성 되어있는
- **기본 내장 app**들에 대한 migration 파일들이 함께 migrate 되기 때문

```
$ python manage.py migrate
Operations to perform:
  Apply all migrations: admin, articles, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying articles.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

<그림17_Django_Model_초기 마이그레이션>

```
INSTALLED_APPS = [
    'articles',
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
]
```

SQLite

SQLite란?

SQLite | **데이터베이스 관리 시스템** 중 하나이며 **Django의 기본 데이터베이스**로 사용됨

- 파일 기반: 데이터베이스가 하나의 파일로 저장되어, 설치/설정 없이 **간편하게 복사/이동/백업** 가능합니다.
- 가볍고 빠름: 별도 서버 없이 파일로 직접 데이터 처리. 소규모 앱이나 모바일 환경에 최적화합니다.
- 높은 호환성: 다양한 운영체제와 프로그래밍 언어에서 폭넓게 사용 가능합니다.

SQLite 주의사항

- db.sqlite3 파일은 Git 등 **버전 관리 시스템에서 관리하지 않는 것이 원칙**
  - 데이터가 변경될 때마다 **파일 전체가 변경됨**
- SQLite 파일은 **로컬 컴퓨터에 저장된 데이터 기록**
- `.gitignore` 파일에 db.sqlite3를 추가하여 Git **버전 관리에서 제외해야 함**

.gitignore
git 버전 관리 시스템에서 특정 파일이나 폴더를 추적하지 않도록 설정하는 파일

<개념4_Django_Model_gitignore>

TIP

gitignore.io란?
- gitignore.io는 개발자들이 .gitignore 파일을 쉽게 생성할 수 있도록 도와주는 웹 서비스
- gitignore.io 에서 Django 를 기술 스택으로 추가한 뒤 .gitignore 파일을 자동 생성하면 db.sqlite3 가 자동으로 포함됩니다.

실습

migrate

- 2665. 장고 model 만들기 예제 1

Admin site

- 2666. 장고 model 만들기 예제 2
- 3230. 상품 관리 시스템 만들기 - 데이터 생성과 관리

Model class

- 3229. 상품 관리 시스템 만들기 - 앱 생성과 모델 정의

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

1 Django에서 모델 클래스는 어떤 클래스의 서브클래스로 정의되어야 하는가?

a) HttpResponse
b) ModelForm
c) models.Model
d) admin.ModelAdmin

2 사용자의 이름을 저장하기에 가장 적절한 필드 타입은 무엇인가?

a) IntegerField
b) CharField
c) BooleanField
d) DateTimeField

3 필드를 필수 입력 항목으로 설정하려면 어떤 옵션을 사용해야 하는가?

a) max_length
b) default
c) null=True
d) blank=False

4 모델 필드에 기본값을 지정하는 옵션은 무엇인가?

a) default
b) unique
c) choices
d) max_length

Test

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

5 날짜와 시간을 모두 저장해야 할 경우 적합한 필드 타입은 무엇인가?

a) DateField
b) TimeField
c) DateTimeField
d) DurationField

6 모델 변경 사항을 반영하여 migration 파일을 생성하는 명령어는 무엇인가?

a) python manage.py runserver
b) python manage.py makemigrations
c) python manage.py migrate
d) python manage.py startapp

7 migration 파일을 실제 데이터베이스에 적용하려면 어떤 명령어를 사용해야 하는가?

a) makemigrations
b) startproject
c) migrate
d) collectstatic

8 다음 중 CharField를 정의할 때 입력 값 길이 제한을 위해 부여할 수 있는 옵션은 무엇인가?

a) null
b) max_length
c) unique
d) default

확인 문제

Q 각 문제에 올바른 답안을 생각해봅시다

9 Django의 기본 데이터베이스 엔진으로 설정된 것은?

a) MySQL
b) PostgreSQL
c) Oracle
d) SQLite

10 Django 모델 클래스 안에서 정의된 각 필드는 어떤 의미를 가지는가?

a) HTML 입력 필드
b) 뷰 함수
c) 데이터베이스 컬럼
d) URL 라우팅

11 Admin 사이트에 모델을 등록하려면 어떤 함수를 사용해야 하는가?

a) include()
b) register()
c) admin.site.register()
d) path()

12 Django에서 객체가 생성된 시점을 자동으로 기록하려면 DateTimeField에 어떤 옵션을 설정해야 하는가?

a) auto_add=True
b) auto_now=True
c) auto_created=True
d) auto_now_add=True

확인 문제

정답 및 해설

1. c) models.Model 2. b) CharField 3. d) blank=False 4. a) default 5. c) DateTimeField 6. b) python manage.py makemigrations 7. c) migrate 8. b) max_length 9. d) SQLite 10. c) 데이터베이스 컬럼 11. c) admin.site.register() 12. d) auto_now_add=True

1. Django의 모델은 `models.Model` 클래스를 상속받아 정의되어야 한다.
2. CharField는 문자열 데이터를 저장하는 데 사용되며, 이름과 같은 짧은 텍스트에 적합하다.
3. `blank=False`는 해당 필드가 빈 값일 수 없음을 의미하여 폼에서 필수로 입력하도록 만든다.
4. `default` 옵션은 객체를 생성할 때 필드에 지정된 기본값을 자동으로 설정해 준다.
5. `DateTimeField`는 날짜와 시간을 모두 저장할 수 있는 필드 타입이다.
6. makemigrations 명령어는 모델의 변경 사항을 기반으로 새로운 `migration` 파일을 생성한다.
7. `migrate` 명령은 `migration` 파일을 바탕으로 데이터베이스를 실제로 변경해 준다.
8. CharField는 문자열 길이를 제한하기 위해 `max_length` 옵션을 지정할 수 있다.
9. Django는 기본 설정에서 SQLite를 데이터베이스 엔진으로 사용한다.
10. 모델 클래스의 각 필드는 실제 데이터베이스 테이블의 컬럼에 해당한다.
11. admin.site.register() 함수를 사용하여 모델을 Admin 사이트에 등록할 수 있다.
12. `auto_now_add=True`는 객체가 처음 생성될 때의 시각을 자동으로 기록해준다. 보통 생성 시점을 저장할 때 사용된다.

활동 정리

핵심 키워드

개념	설명	예시
Model 클래스	DB 테이블 구조를 설계 및 조작하는 청사진 역할의 Python 클래스	`class Article(models.Model): ...`
Model Field	테이블의 열(column)을 정의하며 데이터 **유형**·**제약**을 명시	`title = models.CharField(max_length=10)`
Field Type	DB에 저장될 데이터 종류를 지정하는 클래스	`CharField – 짧은 문자열, TextField – 대용량 텍스트`
Field Option	필드의 동작·제약 조건을 설정	`max_length=10, null=False, default='-'`
Migration	모델 변경 사항을 DB에 반영하는 설계도·변경 이력	`모델 수정 → makemigrations → migrate” 흐름`
makemigrations	모델 변경을 감지해 **migration 파일**(설계도)을 생성	`$ python manage.py makemigrations`
migrate	생성된 migration을 실행해 DB 구조를 실제로 수정	`$ python manage.py migrate`
Admin Site	모델을 등록하면 웹 UI에서 CRUD를 즉시 수행 가능	`admin.site.register(Article) 후 /admin 접속`

<표3_Django_Model_핵심 키워드>

활동 정리

오늘 공부한 내용 요약 및 정리

- Model 클래스 = DB 설계도
  - Article 등 models.Model을 상속해 컬럼을 파이썬 코드로 정의하면, 그 코드가 곧 데이터베이스 테이블 청사진이 된다.
  - "`models.Model` 상속 덕분에 DB 연결·SQL 작성은 Django가 대신하고, 개발자는 테이블 구조 설계에만 집중한다 — 이것이 프레임워크의 기능 제공 방식"
- Django_modelField type & option으로 컬럼 속성 지정
  - CharField, TextField 같은 유형과 max_length, null, default 같은 제약 옵션으로 데이터 형태와 규칙을 명확히 설정한다.
- Django_model마이그레이션 두 단계
  - 모델 변경 ▶ python manage.py makemigrations(설계도 생성) ▶ python manage.py migrate(DB 적용) 순서로 진행하며, 변경 이력은 파일로 관리된다.
- Django_modelAdmin Site로 즉시 CRUD
  - admin.site.register(Article) 후 /admin에 접속하면 브라우저에서 게시글 생성·조회·수정·삭제를 바로 실행할 수 있다.

활동 정리

Django는 파이썬 문법으로 테이블을 만들고 관리합니다.

1. 모델 작성 → 마이그레이션 → DB 반영 전체 흐름을 스스로 설정, 수정 및 확인할 수 있다.
2. **Admin Site**를 통해 코드 변경 없이 데이터를 생성·조회·수정·삭제할 수 있다.
3. **Field type/option** 선택으로 컬럼의 데이터 형식과 제약을 적절히 설정할 수 있다.

- 이처럼, 파이썬 코드만으로 데이터 설계부터 관리까지 한 번에 해결하는 것이 Django Model의 핵심 가치입니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

감사합니다